

Hauling Optimisation

Stockpile → Jetty haul circuit — AI dispatch & bottleneck optimisation

An interactive demo for the OreSight AI platform: a fleet of haul trucks shuttling nickel ore from the port stockpiles to the jetty surge hopper that feeds the barge loadout, with an AI dispatch brain that identifies the binding constraint, re-allocates the fleet, and re-plans live when a disruption hits. Sibling to the Future Scheduling Platform (FSP) and its upstream feeder.

Visual model	Looping route map (animated truck cycle), per the chosen direction
Haul leg	Port stockpile (SP-1 / SP-2) → jetty surge hopper (~4.2 km)
Design system	Reuse OreSight light/dark control-room language from FSP
Sources	Modular Mining DISPATCH, Wenco, Cat MineStar; Samsara / Geotab fleet telematics
Date	13 June 2026

1 · Context & goal

The haul circuit is the **upstream feeder of the FSP chain**. A fleet of nine Komatsu HD785-7 haul trucks (91 t rated) carries ore ~4.2 km from two port stockpiles to the jetty surge hopper that feeds the barge loadout. The delivered rate must track the active barge's loadout demand (~1,900 t/h nominal). The hopper is a small surge bin: if it **starves**, barge loading stalls and the laycan / demurrage exposure FSP already models rises; if it **overflows**, trucks queue and spill. This direct coupling to FSP is the narrative spine of the demo.

The goal of the design is a demo that *feels like one product* with FSP: a live operations screen where the AI does the dispatch thinking — names the binding constraint, re-allocates trucks and loaders, and re-solves on disruption — not a static dashboard.

What the industry does (research synthesis)

- **Real-time truck-loader-dump assignment** is the core of every mining FMS (Modular Mining DISPATCH, Wenco, Cat MineStar). Documented gains: DISPATCH cut truck idle from **210 to 38 h/month** and lifted ton-miles-per-hour **+29%** after dispatch recalibration.
- Systems classify each asset by **state** — loading, hauling, dumping, travelling, waiting/queuing, idle/parked — and colour-code it on a live map. Excessive queuing at load and dump is the headline productivity killer.
- **Match factor** (haul units vs loading units) is the canonical balance lever: ~1.0 balanced, <1 loaders wait (under-trucked), >1 trucks queue (over-trucked). Inadequate tyre **TKPH** is cited as ~62% of truck-bunching causes.
- Fleet telematics dashboards (Samsara, Geotab, Penske-class) lead with a **live map for the dispatcher**, then KPI cards. 2026 best practice: **4 'north-star' KPIs large at the top**, 4–6 supporting metrics in a secondary band; gauges, color-coded KPIs and bubble/colour encoding for urgency.

2 · Metrics to adopt

North-star KPIs (large, top band)

Four headline numbers, mirroring the FSP KPI row. These answer: are we meeting demand, and how efficiently?

Metric	Definition	Unit	Target band
Delivered rate vs demand	Ore reaching the jetty hopper vs the active barge loadout demand	t/h	≥ demand (~1,900)
Match factor	Assigned trucks ÷ trucks needed to keep loaders continuously loading	ratio	0.95 – 1.05
Fleet utilisation	Productive (loading/hauling/dumping) time ÷ available time	%	> 85% (top >78%)
Hopper buffer	Surge-bin level expressed as minutes-to-starve at current demand	% / min	keep > 0; warn <10 min

Supporting metrics (secondary band)

Metric	Definition / why it matters	Unit
Cycle time	Full loop: spot → load → haul loaded → dump → return empty (+ queue/TKPH delay)	min
Queue / wait time	Time trucks wait at loader + hopper; the primary loss to attack	min/cyc
Payload compliance	Loads inside the 10/10/20 policy (mean ≤ rated; ≤10% >1.1×; none >1.2×)	%
TKPH headroom	Tyre tonne-km/h vs rating; margin protects against wet-season heat failures	%
Fuel per tonne	Litres burned per tonne delivered; idle + overload inflate it	L/t
Loader utilisation	Per-loader busy time; reveals loader-bound vs truck-bound state	%
TMPH / productivity	Ton-miles per hour (DISPATCH north-star); +29% achievable headline	idx

Value model (reuse FSP approach): \$ per extra tonne to the jetty + demurrage protected on the coupled barge + fuel saved + idle hours cut, derived from the numbers above — never a canned figure.

3 · Truck-state model & colour coding

Six states, colour-coded consistently across the route map, the dispatch table status dots, and the trend chart. Colours reuse the FSP Gantt block palette so the two demos read as one system.

State	Meaning	Swatch
LOADING	At a stockpile loader (LD-1 / LD-2)	
HAULING (loaded)	On the ramp to the jetty hopper	
QUEUING / WAITING	Bunched at loader or hopper — the loss to minimise	
DUMPING	Tipping into the jetty surge hopper	
RETURNING (empty)	Back to the stockpile	
DOWN / PARKED	Breakdown, refuel, crib break — out of cycle	

Cycle-time anatomy (for the trend / breakdown view)

Spot at loader · load · loaded travel · spot at dump · dump · empty travel · **queue delay** · **TKPH delay**. The queue and TKPH segments are the AI's optimisation targets — surface them explicitly.

4 · Screen layout & components

Follow the FSP page skeleton: breadcrumb → demo-head (title + WITA clock + pitch) → KPI row → dark 'control-room' screen → AI panel + side panels. Components, top to bottom:

#	Component	Spec
1	KPI row	4 north-star cards (delivered t/h vs demand, match factor, fleet utilisation, hopper buffer). Up/down deltas vs plan, like FSP.
2	Constraint (drum) strip	Reuse FSP bottleneck strip: binding constraint = loader / haul-road / hopper / fleet, with utilisation bars and a 'constraint shifted X→Y' indicator on re-dispatch.
3	Looping route map (HERO)	Animated SVG loop (same engine as FSP inload convoys): SP-1/SP-2 loaders → ramp → jetty hopper → return. Truck dots coloured by state; queue badges at loader and hopper; hopper level gauge on the jetty node. ~9 trucks circulating.
4	Disruption controls + free-form	Preset buttons (truck down, loader down, road wet/ramp, demand surge) + NL box ('HT-105 down, ramp wet') → AI re-dispatches. Mirror FSP exactly.
5	Dispatch table	Per-truck rows: id, state dot, assigned loader, last cycle (min), payload (t) vs rated, next AI action. Sortable feel; highlights moved/re-assigned units.
6	AI recommendation panel	renderAIResult-style: headline + fleetActions (reassign/hold/refuel/reroute with $\Delta t/h$) + recommendations + value protected. Live rationale on each re-dispatch.
7	Trend / gauges	Delivered t/h vs demand line over the shift; match-factor dial; hopper-buffer gauge. Chart.js, FSP dark theme.
8	Risk radar + copilot	Optional, as on FSP: forecast wet ramp / tyre TKPH / truck degradation (links to Maintenance demo); grounded Q&A copilot over live fleet state.

Tabs inside the control-room screen (optional): **Fleet board** (route map) · **Dispatch** (table) · **Trends** (charts) — mirroring FSP's UC2/UC1/UC3 tabs.

5 · AI capabilities

- **Dynamic dispatch** — assign trucks to loaders/routes to match hopper demand with minimum queueing (match-factor driven).
- **Bottleneck detection** — name the binding constraint (loader / haul-road / hopper / fleet) and the marginal t/h of relieving it.
- **Payload & tyre/fuel optimisation** — 10/10/20 policy compliance, TKPH headroom, eco-speed vs cycle-time trade-off.
- **Queue smoothing & refuel/crib staggering** — keep loader feed and hopper level above the demand line.
- **Demand coupling to FSP** — when a barge is laycan-critical, surge the fleet to build hopper buffer ahead of its load window (and ahead of a forecast swell).
- **Disruption re-dispatch** — truck down, loader breakdown, wet ramp, demand surge → live re-solve with rationale (FSP-style).
- **Anomaly detection** — a cycle-time outlier flags a truck → hands off to the Maintenance demo.

6 · Claude prompt (drop-in for server.js)

Same house style as the existing OreSight prompts (persona → exact JSON schema → domain rules → JSON-only). One endpoint, `/api/haul/analyze`, handles both 'optimise current state' and 'disruption re-dispatch' via an optional scenario in the user message. The `{headline, recommendations[], valueImpactUSD, narrative}` core reuses `renderAIResult`; the specialised fields drive the bespoke UI. Ship with a deterministic `haulFallback()` so it runs with no API key, exactly like the others.

```
HAUL_PROMPT = `You are OreSight AI's haulage dispatch optimiser for the port haul
circuit at a nickel laterite operation in Morowali, Central Sulawesi. A fleet of
nine Komatsu HD785-7 haul trucks (91 t rated payload) shuttles ore ~4.2 km from two
port stockpiles (SP-1 saprolite, SP-2 limonite blend) to the jetty surge hopper that
feeds the barge loadout. Two loaders work the stockpiles: LD-1 (Cat 992K wheel loader)
and LD-2 (Hitachi EX1900 excavator). Two 12-hour shifts (07:00/19:00 WITA). The
delivered rate must track the active barge loadout demand (~1,900 t/h): the hopper is a
small surge bin - if it starves, barge loading stalls and laycan/demurrage exposure
rises; if it overflows, trucks queue and spill.

 
Given a JSON snapshot of the haul circuit (fleet status & cycle times, loader status,
haul-road/weather state, hopper level, and the barge loadout demand it must serve) -
and optionally a disruption scenario - produce a dispatch optimisation. Return JSON
with EXACTLY this structure:

 
{
  "headline": "<one-line conclusion: current delivered rate vs demand>",
  "bindingConstraint": "loader" | "haul-road" | "jetty-hopper" | "fleet",
  "constraintNote": "<why this is the binding constraint, one line>",
  "currentRateTph": <number, ore delivered to the hopper now, t/h>,
  "optimisedRateTph": <number, achievable after the actions; >= currentRateTph>,
  "matchFactor": <number; 1.0 balanced, <1 loader-starved, >1 trucks queue>,
  "fleetActions": [
    {"unit": "<HT-104 / LD-2>", "action": "reassign|hold|release|refuel|reroute|payload",
    "detail": "<short specifics>", "deltaTph": <t/h gained or recovered>}
  ],
  "recommendations": [
    {"action": "<dispatch / road / payload / sequencing action>",
    "impact": "<quantified: t/h, queue-min, fuel-L, US$>", "timeframe": "<when>"}
  ],
  "forecastNext4h": {"tonnesToJetty": <int>, "hopperRisk": "starve|balanced|overflow",
  "hopperNote": "<one line>", "fuelLitres": <int>},
  "valueImpactUSD": <integer, annualised value of the optimisation set>,
  "narrative": "<3-4 sentences linking the moves to the barge demand & constraint>"
}

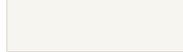
 
Domain rules:
- Match factor MF ~ (assigned trucks)/(trucks needed to keep loaders loading). MF<0.9
-> loaders wait (under-trucked: add/route trucks); MF>1.1 -> trucks bunch (over-
trucked: hold/redeploy). Drive toward 0.95-1.05.
- The hopper is the pacemaker: optimisedRateTph tracks demand, not wasteful excess. If
demand exceeds capacity, name the binding constraint and the marginal t/h to relieve it.
- Respect limits: 2 loaders, 9 trucks, 91 t rated. Apply the 10/10/20 payload policy
(mean <= rated; <=10% of loads >1.1x; none >1.2x). Watch tyre TKPH in wet-season heat.
- Wet-season roads: watering cuts dust but reduces traction on the ~8% ramp; recommend
watering passes and speed limits that protect cycle time without spillage.
- Stagger refuels and crib breaks so loader feed and hopper level never drop below the
barge demand line.
- If a barge is flagged laycan-critical, prioritise building hopper buffer ahead of its
load window - surge the fleet even at slightly higher fuel per tonne.
- deltaTph across fleetActions should reconcile with (optimisedRateTph - currentRateTph).
- 3-5 fleetActions and 3-4 recommendations, each concrete and quantified.

 
IMPORTANT: Return ONLY valid JSON, no other text.`;
```

Companion (optional, mirrors FSP): a small HAUL_PARSE_PROMPT behind /api/haul/parse turns a free-text note ('HT-105 down, ramp wet') into a structured disruption the engine re-dispatches.

7 · Design system (reuse OreSight tokens)

Build in the existing styles.css language so the demo is visually identical to FSP. Core palette:

					
Accent / brand #15803d	Control-room screen #10150f	App background #f6f5ef	Warning / queue #b45309	Hauling / transit #0e7490	Down / critical #b91c1c

- **Type:** Inter (UI) + JetBrains Mono (labels/data), already loaded. Mono uppercase for axis/stage labels.
- **Components to reuse verbatim:** kpi-card, fsp-screen / fsp-tab / fsp-status, bottleneck-strip, option-cards, gantt convoy animation engine, ai-panel + renderAIResult, risk-item, copilot, freeform-row.
- **New components:** looping-route SVG (loaders, ramp, hopper-with-gauge nodes), match-factor dial, hopper-buffer gauge, dispatch-table.
- **Motion:** truck dots animate continuously (requestAnimationFrame, like FSP convoys); on re-dispatch, re-assigned trucks pulse and the constraint strip updates with the X→Y shift.

8 · Build notes

- **Page:** mining-demo/public/haul.html + js/haul.js; **endpoint:** /api/haul/analyze (+ optional /api/haul/parse) in server.js; **fallback:** data/haul.js (deterministic).
- **Deterministic engine is the source of truth** for the on-screen fleet (always re-dispatches visibly); AI parses free-text and writes rationale on top — works fully with no API key.
- **Matrix wiring:** promote the existing 'Haul Optimisation' (or 'Dispatch Optimisation') cell on the value-chain landing page to a LIVE DEMO linking to /haul.html.
- **Nav:** add a 'Hauling' link in the shared nav, between Scheduler (FSP) and Control Tower.

Sources

- Komatsu / Modular Mining DISPATCH case study (idle 210→38 h/mo; +29% TMPH) — komatsu.com/en-us/case-studies/dispatch-fleet-management-system-helps-mine-optimize-its-haulage
- Wenco Mining Fleet Management — wencomine.com/our-solutions/mining-fleet-management
- Cat MineStar Solutions — cat.com/en_US/by-industry/mining/minestar-solutions.html
- DISPATCH Fleet Management — mining-technology.com/products/dispatch-fleet-management/
- Improving Shovel-Truck Productivity (match factor, UMaT) — conference.umat.edu.gh
- Truck Haulage: Selection, Cycle Time & Output (DGMS) — onlineminingexam.graphy.com
- Haul Trucks Queuing Prediction in Open Pit Mines (A. Soofastaei) — linkedin.com/pulse
- Fleet Management Dashboard KPIs & best practice — superblocks.com/blog/fleet-management-dashboard; geotab.com/blog/fleet-management-dashboard; samsara.com/products/telematics
- Modern Fleet Management UI/UX (north-star + supporting KPI pattern; utilisation >78%) — heavyvehicleinspection.com